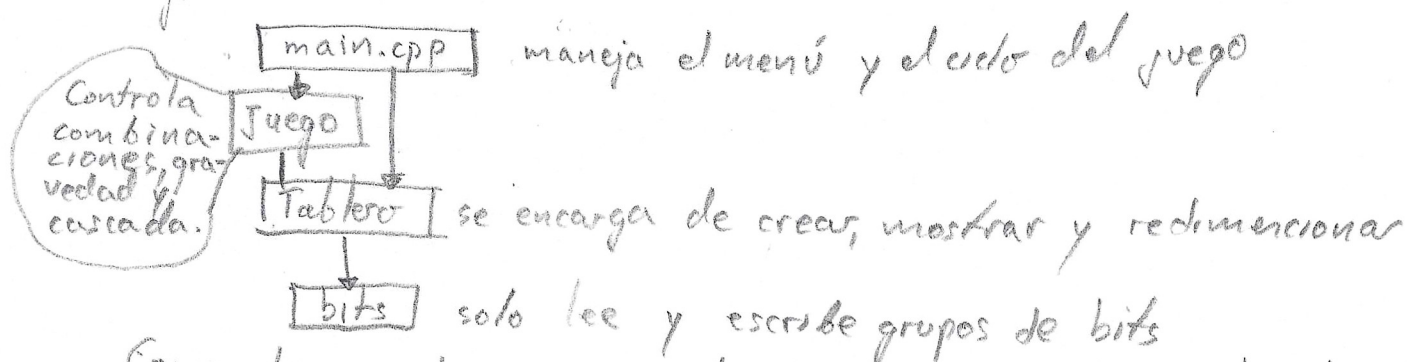


Bloque 4 Diseño

Arquitectura en capas

Diagrama



Como la regla es que cada capa solo conoce la de abajo. Es decir, que el módulo bits no conoce lo que es una ficha, por ejemplo; el tablero no sabe que es una combinación, y el juego no toca bytes ni desplazamientos. Permitiendo probar cada parte por separado.

El problema es que no se puede usar struct, entonces filas y columnas deben pasarse sueltas entre funciones. Se pueden manejar manejar mejor!!

- Las funciones que agregan o eliminan filas o columnas devuelven el puntero, para poder redimensionar y el que llama debe quedarse con el puntero correcto.
- Se pasan las dimensiones por referencia (`int &filas`), para que los cambios se reflejen afuera

Tipos de datos y estado del juego

tipos básicos:

- unsigned char para el tablero: es el tipo más pequeño direccionable y no tiene signo, no se corrompen los bits al hacer corrimiento o ejecutar.
- unsigned char para una ficha: solo necesita 3 bits.
- int para filas, columnas e índices: es el tipo natural para cantidades pequeñas y positivas
- unsigned char para las marcas: un mismo empaquetado, con 1 bit por casilla

El estado del juego se guarda en variables static: eliminaciones, fichas eliminadas, combinaciones, cascadas, puntaje y bytes reservados. Se espera que las variables conserven su

valor entero llamado y no haya que pasarlo como parámetros en todas las funciones.

La puntuación está definida como:

$$\text{puntos} = 10 \times \text{fichas eliminadas} \times \text{número de vuelta}$$

para premiar las jugadas encadenadas, porque una cascada en la tercera vuelta vale más que en la primera.

También se puede usar const en parámetros que solo se leen y para constantes en lugar de #define, ya que en C++ las const tienen tipo y enlace interno.